Shape-Aware Three-Way Merge for RDF Knowledge Graphs

The schema defines the conflict: an implementation and evaluation

Steve Brown*

30 August 2026

Abstract

Version control works for source code because its merge understands the medium: lines. Knowledge graphs shared between people and software agents are not lines, and merging their serialisations makes conflicts out of ordering and misses the ones that matter. We describe and evaluate a three-way merge for RDF in which the *schema* decides what a conflict is: the merge is set algebra over canonical triples, and a slot is contended only where a SHACL shape declares that it can hold at most one value. Multi-valued predicates union; functional predicates with divergent values are handed to a person. The same shapes graph then validates the merge result, so the schema that defines conflicts also audits their resolution.

We do not claim the first three-way merge for RDF. Git-backed RDF versioning with merge strategies is established prior work, and the closest neighbour — Quit Store's Context Merge — is included here as a baseline that this operator extends rather than replaces. Our contribution is an implementation in a production multi-agent store and an evaluation of what schema-derived conflict semantics buys against that neighbour and against six other strategies.

On a synthetic divergence benchmark over five seeds, the shape-aware operator raised 33 conflicts, all true positives, with no false positives, no triples lost, none fabricated, and no SHACL violations admitted. The context-overlap baseline detected the *same* 33 conflicts and asked 845 questions to do it. We also report a limit no schema removes: when two sides mint different names for one entity, the divergence is invisible to any triple-level operator, including this one — 21 of 54 synthetic conflicts and 93 of 93 alias repairs in a recorded corpus. Replaying 105 real repairs and 939 real duplicate-value incidents from a production graph against the shipped operator reproduces both the blindness and the benefit on data nobody generated for the purpose.

1 Introduction

A knowledge graph that more than one writer can edit will diverge, and something has to put it back together. The obvious move is to reuse the tooling that already solves this for source code: keep the graph in a file, keep the file in git, and let a line merge reconcile two versions.

*ORCID `0009-0009-1720-1785`. With implementation and drafting assistance from Claude (Anthropic). **Artifact availability:** the merge operator, the divergence benchmark (`benchmark/mergebench/`) and the recorded-divergence replay corpus (`benchmark/replay/`) behind every number reported here are in the development repository, `github.com/scbrown/quipu`. Each result table names the command that regenerates it.

That move fails in a specific and instructive way. A line merge understands a document as an ordered sequence of lines, and a graph is neither ordered nor a sequence. Two writers who add unrelated facts about the same entity produce adjacent edits and a textual conflict. Two writers who re-serialise the same unchanged graph produce a diff of everything. Meanwhile a writer who sets a second value on a property the ontology says holds exactly one — the change that genuinely cannot be reconciled without a decision — produces a clean, conflict-free merge, because the two values live on different lines and nothing in the file says they compete.

The failure is not that line merge is bad at graphs. It is that conflict is a *semantic* property of the data model, and a line merge is being asked to infer it from a syntactic one. Our position is that an RDF merge should take conflict from the place the data model already defines it: the schema.

This paper. We describe an operator with two halves. Triples are canonicalised and merged as sets, so serialisation order cannot manufacture a conflict. A SHACL shapes graph then decides which slots the set algebra is allowed to settle on its own: where a shape declares `sh:maxCount 1` and the two sides carry different values, the operator refuses and records a decision; everywhere else, it unions. Because the shapes are already present — they are what validates writes into the store — the merge policy is not a second configuration to keep in step with the ontology. It *is* the ontology.

What we are not claiming. Three-way merge over canonicalised RDF is not new, and we searched before writing. Quit Store [1] places RDF under git with merge strategies, including a Context Merge that reconciles by node overlap; TerminusDB offers diff and patch over a versioned graph; RDF Patch and LD Patch standardise change description; RDFC-1.0 gives canonical labelling for graphs with blank nodes, which we build on rather than beside. To our knowledge — and scoped to that search — what has not been reported is an evaluation of *SHACL-derived* conflict semantics against those neighbours in a deployed system. That evaluation is the contribution, and the framing of the operator as an extension of Context Merge rather than a replacement for it is deliberate.

Contributions.

1. A three-way merge operator over canonical triples whose conflict predicate is derived from SHACL cardinality, with post-merge validation against the same shapes, implemented in a production store (§3, §4).
2. A divergence benchmark with an oracle recomputable by a reader from published data alone, scoring eight strategies under one accounting contract, and a result that contradicts one of our own hypotheses (§5).
3. A replay of recorded multi-agent divergence from a production graph against the shipped operator, which reproduces the synthetic arm's blind spot on real data and quantifies the benefit on a class that production surfaced nothing of (§6).
4. A stated limit — alias divergence is invisible to triple-level merge — reported as a measured false-negative column rather than excluded from the benchmark (§7).

2 Background

The unit of change. An RDF graph is a set of triples. Two serialisations of one graph may differ in every byte — Turtle groups by subject, N-Triples does not, and neither fixes an order within a group. A diff over serialised text therefore reports differences that do not exist in the data. Taking the triple as the unit of change removes that entire category: a three-way merge becomes set algebra over (G_b, G_o, G_t), and no result depends on how any side chose to write its file.

Blank nodes are the exception, since they have no stable name across serialisations. RDFC-1.0 (formerly URDNA2015) assigns canonical labels and makes graph-level equality decidable. Our operator sits on top of that substrate; the benchmark in §5 uses ground graphs only, so it does not exercise blank-node canonicalisation and is not evidence about it.

Where conflict comes from. Set algebra alone answers only part of the question. If both sides add a triple, the union holds both. That is correct for a person's email addresses and wrong for their date of birth, and nothing in the triples distinguishes the two cases. The distinction is in the schema: a SHACL shape with `sh:maxCount 1` on a property path states that the slot holds at most one value. A merge that unions into such a slot has not resolved the divergence; it has produced a graph that violates the very constraint the store enforces on ordinary writes.

This gives a conflict predicate with three properties we care about. It is *declared*, not inferred from edit patterns. It is *already present*, because the shapes are what the store validates against. And it is *auditable*, because the same shapes graph can be run over the merge output — so the definition of a conflict and the test for a bad resolution are one artefact rather than two that can drift apart.

Shares. The operator reconciles *share bundles*: a directory holding a canonical N-Triples export, the shapes that govern it, and a manifest carrying a content hash of the graph and the identifier of the share it was derived from. The parent pointer is what supplies G_b, so fork-point anchoring is by content hash rather than by wall-clock time or by a centrally-issued sequence number. Two peers who have never spoken can therefore establish a common ancestor from the bundles alone.

3 The operator

Let G_b, G_o and G_t be sets of canonical triples, and let S be a SHACL shapes graph. A *slot* is a pair (s, p) of subject and predicate; we write $G[s, p] = \{\, o : (s, p, o) \in G \,\}$ for the values a graph carries in a slot. The slot is the unit of conflict because it is the unit `sh:maxCount` is declared over.

Additions and removals. Each side's change relative to the base is $A_x = G_x \setminus G_b$ and $R_x = G_b \setminus G_x$ for $x \in \{o, t\}$. These are exact: no heuristic decides whether a triple was "modified" rather than removed and re-added, because at triple granularity there is no such distinction to make.

Functional slots. Let $max_S(s, p)$ be the maximum cardinality S declares for p on s, or ∞ where none is declared. A slot is *contended* when

$$max_S(s,p) = 1 \ \wedge \ G_o[s,p] \neq G_t[s,p] \ \wedge \ G_o[s,p] \neq G_b[s,p] \ \wedge \ G_t[s,p] \neq G_b[s,p].$$

Both sides must have moved. If only one side changed a functional slot, the other side's value is still the base value, there is no disagreement, and the change applies — a merge that stopped to ask about it would be charging a person for a decision nobody needs to make.

The merge. If any slot is contended, the operator returns `conflicts` and writes nothing; each contended slot is reported with its base, ours and theirs values, and the declared cardinality that made it a conflict. Otherwise the result is

$$G_m \ = \ (G_o \cup G_t) \setminus (R_o \cup R_t),$$

a retraction on either side being honoured against the other side's retained triples. The transaction records *both* parent share identifiers, so the merge is itself provenance: the lineage of a merged graph is inspectable in the same way a merge commit's is.

Two properties worth stating. The operator never lands a value for a slot it flagged — a conflicted merge writes nothing at all — so it cannot score well by both refusing a decision and quietly resolving it. And because G_m is defined by set operations over canonical triples, it is commutative in G_o and G_t and independent of serialisation.

Post-merge validation. G_m is validated against S. This is not belt-and-braces: it is the falsifiable form of the design claim. If conflict semantics really are derivable from cardinality, then a merge that consulted the shapes should not be able to produce a graph those same shapes reject. Any violation in G_m is corruption the operator admitted without charging anyone for it, and §5 reports that column for every strategy precisely so the claim can fail.

4 Implementation

The operator ships in Quipu, a governed bitemporal RDF store, as `share_merge` (approximately 480 lines of Rust). It exposes two entry points. `status` loads an incoming bundle, resolves the common ancestor through the manifest's parent pointer, and reports what each side added and removed together with the contended slots — without writing anything, so a peer can be inspected before it is merged. `merge` performs the algebra of §3 and commits, recording both parent share identifiers on the transaction.

Three implementation choices are worth naming because they affect what the evaluation can claim.

The shapes come from the bundle, not the store. Cardinality is read from the shapes carried inside the share being merged, not from the local store's copy. A peer's constraints therefore travel with its data, and merging does not silently reinterpret their graph under our schema.

Conflict is reported at slot granularity, with its evidence. A `DecisionRecord` carries the subject, the predicate, the declared `maxCount`, and the base, ours and theirs value sets. The person resolving it is not told that a conflict exists and left to find it; they are told which constraint made it one and what the three candidate states were.

The evaluated operator is the shipped one. The benchmark of §5 contains its own reference implementation of every arm, including ours, so that all eight are scored under one accounting contract. The replay of §6 deliberately does *not*: it drives `share_merge::{status,merge}` through real bundles built by the production share path. This matters because a benchmark's copy of an operator can agree with a benchmark's copy of an oracle for reasons that have nothing to do with the deployed system.

5 Arm A: a synthetic divergence benchmark

Design. One seeded base graph over a synthetic vocabulary is copied twice and edited independently — separate random streams, so neither side's edits can be a function of the other's — under a six-operation model: assert, retract, re-describe, re-type, alias-mint, node-add. An overlap parameter controls how often an edit targets a contended quarter of the entity set, so collision probability is set directly rather than left to chance. Eight strategies then merge (G_b, G_o, G_t) and are scored against one oracle. There is no language model anywhere in the loop; the writers are deterministic, so a run is its own ground truth.

The oracle is a function of published data. `ground_truth` takes the three graphs and the shapes and nothing else — no edit log, no generator intent — so a reader holding only the published inputs recomputes it exactly, and a generator bug cannot hide inside it. An instrument control asserts precisely that.

The accounting contract. Every arm answers two questions whose answers are in tension. *Human decisions* is the number of slots it refused to settle; a refused slot is held at its base value in the output, so no arm can flag a conflict *and* land a value for it. *SHACL violations* counts violations in what it landed automatically, against the same shapes that defined the conflicts — corruption it admitted without charging anyone. Either can be driven to zero by sacrificing the other, so neither is reportable alone. *Triples lost* and *triples spurious* are the third leg, and they catch the arm that admits no corruption and asks no questions because it discarded one side's work.

Results. Table 1 aggregates five seeds $(1, 7, 42, 99, 2026)$ at 200 entities, 400 edits per side and overlap 0.5. Regenerate with `just bench merge -seed <N>`; instrument controls with `just bench merge -selftest` (7 properties, non-zero exit on any failure).

Three readings matter.

The comparison with the nearest neighbour is not about detection. Summed over the five seeds the oracle contains 54 conflicts, of which 21 are alias-mint and 33 are visible at triple level. SHAPE-AWARE raised exactly those 33, with zero false positives at every individual seed. CONTEXT-MERGE raised the *same* 33 — identical recall, seed by seed — and 812 others, for 845 questions in total. The schema does not find conflicts the node-overlap heuristic misses.

arm	decisions	precision	recall	SHACL	lost	spurious
`git-turtle-reserialized`	681–794	0.006–0.011	0.36–0.80	0–1	266–299	62–71
`git-turtle-stable`	41–59	0.049–0.119	0.20–0.70	0–3	39–57	12–24
`git-canonical`	106–144	0.028–0.057	0.30–0.70	0–3	95–128	18–31
`union`	0	—	0.00	53–72	2–7	112–136
`lww-theirs`	0	—	0.00	0–1	7–16	4–10
`triple-3way`	0	—	0.00	3–6	2–7	7–17
CONTEXT-MERGE	159–182	0.025–0.046	0.50–0.80	0	128–150	63–72
SHAPE-AWARE	4–8	**1.000**	0.50–0.80	**0**	**0**	**0**

Table 1: Eight strategies, five seeds, ranges across seeds. Recall is bounded above by 1 minus the alias-mint fraction for *every* arm (§7). The line-merge arms shell out to real `git merge-file`; where git is absent they report *unavailable*, never zero.

It distinguishes the ones that need a person from the 812 that do not, which is a 25.6× difference in the quantity the operator is actually spending: attention.

The zero columns are not all worth the same. `union`, `lww-theirs` and `triple-3way` each report 0 human decisions, and none of them resolved anything. `union` admitted 53–72 SHACL violations. `lww-theirs` admitted almost none — and silently dropped 7–16 triples and fabricated 4–10, which is what last-writer-wins looks like when you count what it discarded rather than what it complained about. Only SHAPE-AWARE holds all three at zero while still asking.

A hypothesis of ours did not survive. We predicted that canonicalisation alone would materially beat non-canonical line merge. Against the re-serialised arm it does: unparseable output lines go from 116–169 to 0. Against a *stably* serialised Turtle file it does not. `git-canonical` raises *more* conflicts (106–144) than `git-turtle-stable` (41–59), because sorting scatters each subject's triples into one sorted run where two sides' additions land adjacent and collide, whereas subject-grouped Turtle absorbed them into a block. The honest statement is narrower than the hypothesis: canonicalisation buys syntactic integrity and independence from serialisation order, and it does not by itself reduce conflict count. It is a trade, and we report it as one.

The circularity, stated rather than managed away. SHAPE-AWARE and the oracle both read the same shapes graph, and for the triple-visible classes their rules coincide by construction. SHAPE-AWARE's precision of 1.000 on this arm is therefore a property of the design, not evidence about it. We do not repair this by making the oracle cleverer, which would only move the coincidence somewhere less visible. We handle it by reporting per-class results so a reader can see which rows are definitional, by confining this arm's contribution to the *cost of the alternatives*, and by taking correctness evidence from §6, where the operator under test is the shipped one and disagreement is a finding.

6 Arms B and C: recorded divergence, shipped operator

The synthetic arm can measure what the alternatives cost. It cannot be evidence that our operator is correct, because its oracle and its operator share a definition. This arm removes

scenario	decisions	historical	outcome
`alias-id-form`	0	46	merged — **false negative**
`alias-semantic`	0	47	merged — **false negative**
`comment-double`	939	0	conflicts — raised as decisions
`sameas-repair`	0	20	merged — repair survives (wanted)

Table 2: Recorded divergence replayed against the shipped operator. 12 pairs (6 id-form, 6 semantic) were excluded as *chained* — sharing an endpoint with another pair — because including them measures an incidental collision on the shared endpoint rather than alias detection.

that circularity twice over: the corpus is *recorded* production divergence rather than generated, and the operator under test is the shipped `share_merge` driven through real bundles. Here a disagreement between operator and expectation is a finding rather than a harness bug.

The corpus. Extracted from a live multi-agent knowledge graph in which software agents and people had been writing concurrently for months, and committed as a fixture. Entity names are replaced by random labels and the mapping is discarded at build time; structure, cardinality and class membership are preserved exactly, and injectivity is verified over the 1,076 names. Two divergence classes were recorded.

Alias-mint: two names silently minted for one entity, later repaired by a person asserting `owl:sameAs`. There are 171 raw `sameAs` edges, which reduce to 105 distinct undirected repairs (66 edges were one repair recorded twice). Each is one repair a person actually performed — the manual-effort baseline. **The class splits, and the split matters**: 52 of the 105 are *id-form*, two spellings of the same commit identifier, which a normalisation rule could retire without asking anyone; the other 53 are two English phrasings of one concept, where nothing but a person can decide. Reporting 105 as one undifferentiated "human decisions" figure would overstate the irreducible cost roughly twofold.

Comment-double: a `sh:maxCount 1` description predicate that the production append path doubled instead of replacing — 939 affected subjects carrying 1,784 excess values.

Results. Table 2 reports the replay. Regenerate with `cargo run -example replay -features shacl`.

The operator is blind to the entire alias class: 0 of 93. This reproduces the synthetic arm's 0 of 4 on data nobody generated for the purpose, and it is the honest false-negative column. Two names for one entity is invisible to *any* triple-level operator, shape-aware included; no schema fixes it, because it is a limit of the medium the operator works in.

The operator raises 939 decisions on a class production surfaced none of. The same class produced 1,784 silent excess values through the append path. This is a counterfactual and must be read as one (§7).

A repair survives a concurrent edit. `owl:sameAs` carries no cardinality constraint, so a repair asserted on one side unions rather than racing an edit to the aliased node on the other. A merge that could drop repairs would make the repair path itself unreliable, so this outcome is the wanted one rather than an incidental one.

The negative control. Rerunning with the description predicate's `maxCount` unbound takes `comment-double` from 939 decisions to 0, everything merging cleanly. The decisions are therefore *shape-derived* — not manufactured by the harness — and this control arm doubles as the closest thing to a simulation of the production append path: it reproduces the doubling. That is the paper's thesis stated as an experiment. Unbind the constraint and the conflict ceases to exist; the schema is what made it one.

Arm C: share, diverge, reconnect. A case study walks the full lifecycle on a real bundle and verifies all eight hops: share a base bundle; a peer publishes a divergent share naming it as parent; we edit locally; status before reconnect reports `diverged` with +40 triples each side and 0 conflicts; the merge asserts 40 and retracts 0; the transaction records two parents; both sides' work is present afterwards (40/40 each); and status after reconnect reports nothing outstanding. One detail is worth carrying: the graph hash is stable across runs while the share identifier is not, because the manifest embeds a timestamp. Fork-point anchoring depends on the graph hash, so lineage is unaffected — but a share identifier must not be used as a content identity.

7 Limitations

Alias divergence is invisible, and that is a property of the medium. When two writers mint different names for one entity, the graphs disagree about the world while agreeing about every triple. No triple-level operator can see it, so a recall of 1.000 is not achievable by anything in Table 1. We left the class in the benchmark deliberately: it is the most common real divergence defect in the recorded corpus, and excluding it would have produced a shape-aware arm scoring 1.000 on both precision and recall as an artefact of what we chose to measure. Resolving it needs entity resolution — a different kind of system, working from labels and neighbourhoods rather than from triples, and one whose errors are not obviously cheaper than the ones it prevents.

The 939 **is a counterfactual.** Those doublings arose from sequential re-posts through a single write path, not from a share, a divergence and a merge. The supported claim is narrow: had the same edits arrived as a divergence and been reconciled by this operator, each would have been surfaced as a decision instead of silently doubling. It is *not* a claim that the operator was running and prevented 1,784 doublings.

Comment bodies in the corpus are synthetic. Real values are prose about a deployment. Only the multiplicity is real — which is all the merge reasons about, but it does mean the corpus cannot support any claim about value content.

A conflict blocks the whole merge, not the conflicting slot. The operator returns `conflicts` with nothing asserted. On a slice like the 939-subject one, a single reconnect writes nothing until every contended slot is resolved. This is the conservative choice and it is a real usability cost; partial application with a held-back conflict set is future work.

Ground graphs only. The benchmark's graphs contain no blank nodes, so canonicalisation there reduces to sorting. RDFC-1.0 is the substrate we build on and is not exercised by this arm; nothing here is evidence about blank-node canonicalisation.

Timing is indicative. One machine, one process, single runs. The line-merge arms include process spawn for `git merge-file`, of order 5 ms, which dominates their timing at every size measured; those columns are not comparable with the in-process arms and we do not report them as if they were.

Slot granularity is an assumption. A conflict is reported over a $(subject, predicate)$ slot because that is the unit `sh:maxCount` is declared over. An operator working at another granularity would need a different scorer, and the comparison in Table 1 would not transfer unchanged.

One deployment. The recorded corpus comes from a single production store. Its class distribution — alias-mint and functional-value doubling dominating — may not generalise, and we make no claim about divergence rates in other systems from it.

8 Related work

Git-backed RDF versioning. Quit Store [1] is the closest neighbour and the reason this paper makes no priority claim. It places RDF under git — the system's own gloss on its name is "Quads in Git" — versions quads with commit-level provenance, and supplies merge strategies, including a Context Merge that identifies conflicts by node overlap and hands them to a person. The triple-set framing of three-way merge is present there; we are not introducing it.

Our delta is where the conflict predicate comes from. Context Merge derives contention from the *shape of the change*: it marks the subject and object of every added or removed statement with the commit it came from, and a node marked by both sides becomes a conflict. We derive it from the *shape of the data*, as declared in SHACL. The difference is a deliberate design choice on their part rather than an oversight, and they state it: Quit Store declines to encode semantic rules and relies on the pure RDF data model instead, and it presents Context Merge as a *supervised* approach to identifying conflicts rather than as a decision procedure. Our claim is correspondingly narrow. SHACL is exactly the source of semantic rules they chose not to hard-code, and what lets us consult it is circumstance rather than insight: in a shapes-governed store those rules are already present, already authored by the people who authored the data, and already the contract the graph is validated against.

The consequence is measurable rather than rhetorical, and it is the central result of §5: across five seeds the two approaches detect an identical set of 33 triple-visible conflicts, and the context heuristic asks 845 questions where cardinality asks 33. Overlap is a good proxy for "these edits are related", which is what Context Merge sets out to find. It is a poor proxy for "these edits cannot both be true", which is the smaller question that actually needs a person — and Context Merge does not claim to answer it. The 845 against 33 therefore measures two different questions being asked, not one strategy failing at the other's task: it quantifies what the extra information buys, and that information is only available to a merge that has a shapes graph to consult. We position this operator as an extension of that line of work: the algebra is theirs, the conflict predicate is the contribution.

Diff and patch over versioned graphs. TerminusDB [6] provides diff and patch over a versioned graph store. RDF Patch and LD Patch standardise the description of change sets. These solve the representation and transport of a delta; they are complementary to, and do not decide, what makes two deltas irreconcilable.

Canonicalisation. W3C RDF Dataset Canonicalization [4] (RDFC-1.0, formerly URDNA2015) gives deterministic labelling of blank nodes and hence decidable graph equality. We build directly on it: it is what makes "the same triple" well defined across two independently serialised bundles, and it is a substrate rather than a competitor. R43ples [2] and archiving systems such as OSTRICH [5] address efficient storage and querying of revision histories, a different problem from reconciling two live divergent copies.

Convergent replicated types. CRDT constructions for RDF, such as SU-Set [3], guarantee convergence without coordination. They and this operator make opposite trades deliberately. A CRDT resolves every case automatically, which requires that some rule always picks a winner; for a functional predicate whose two values are both asserted in good faith, any such rule silently discards somebody's knowledge. We would rather stop and ask, and accept that this means a merge can fail. Where automatic convergence is required, a CRDT is the right tool and this is not; where the graph is a shared record of what is believed to be true, a silently dropped assertion is the more expensive failure. Operational transformation has the same shape of trade in a different lineage.

Schema-aware merging elsewhere. Ontology alignment and matching address a related question — reconciling different conceptualisations — but operate on schema-to-schema correspondence rather than on three-way reconciliation of instance data under a shared schema. Structured merge tools for source code (semi-structured and AST merges) share our motivation exactly: raise the merge above lines to the level at which the medium defines conflict. Our claim is the RDF instance of that principle, with SHACL supplying the structure.

9 Conclusion

Merging knowledge graphs by merging their text asks a line-oriented tool to infer a semantic property, and it fails in both directions: it manufactures conflicts from serialisation order and misses the divergences that genuinely cannot be reconciled automatically. Taking the canonical triple as the unit of change removes the first failure entirely. Taking the conflict predicate from a SHACL shapes graph addresses the second, and does so without adding a policy that has to be maintained alongside the ontology — because it *is* the ontology, and the same shapes then audit the result.

The evaluation supports a narrow claim, which is the one we make. Against the nearest neighbour, schema-derived conflict semantics do not detect more: they detect the same 33 triple-visible conflicts across five seeds while asking 33 questions rather than 845, with no triples lost, none fabricated and no constraint violations admitted. Replaying 105 real repairs and 939 real duplicate-value incidents against the shipped operator reproduces that benefit on data nobody generated for the purpose — and reproduces the blind spot too: alias divergence is invisible to triple-level merge, 0 of 93, and no schema will fix it.

Two directions follow. Partial application would let a merge land its uncontended slots and hold back only the contended ones, removing the cost noted in §7. And the alias class, which dominates the recorded corpus and which this operator cannot see, is where entity resolution would have to meet merge — with the open question being whether its errors are cheaper than the divergence it repairs.

References

[1] Natanael Arndt, Patrick Naumann, Norman Radtke, Michael Martin, and Edgard Marx. Decentralized collaborative knowledge management using Git. *Journal of Web Semantics*, 54:29–47, 2019.

[2] Markus Graube, Stephan Hensel, and Leon Urbas. Open semantic revision control with R43ples. In *Proceedings of the 12th International Conference on Semantic Systems (SEMANTiCS)*, 2016.

[3] Luis-Daniel Ibáñez, Hala Skaf-Molli, Pascal Molli, and Olivier Corby. Synchronizing semantic stores with commutative replicated data types. In *Proceedings of the 21st International Conference on World Wide Web (WWW '12 Companion)*, pages 1091–1096, 2012. Defines SU-Set. Extended version: Ibáñez et al., Live Linked Data: Synchronising Semantic Stores with Commutative Replicated Data Types, *International Journal of Metadata, Semantics and Ontologies* 8(2):119–133, 2013, DOI:10.1504/IJMSO.2013.056605.

[4] Dave Longley and Manu Sporny. RDF dataset canonicalization (RDFC-1.0). W3C Recommendation, May 2024.

[5] Ruben Taelman, Miel Vander Sande, and Ruben Verborgh. OSTRICH: Versioned random-access triple store. In *Companion Proceedings of The Web Conference 2018*, 2018.

[6] TerminusDB: a versioned graph database. Software, 2024.